Optimization Techniques for Multi Cooperative Systems MCTR 1021
Mechatronics Engineering
Faculty of Engineering and Materials Science
German University in Cairo

TITLE

By

TeamNum
Member1
Member2
Member3
Member4
Member5

Course Team

SUPERVISOR1
SUPERVISOR2
SUPERVISOR3

October 15, 2025

This is to certify that:

(i) the report comprises only my original work toward the course project,

(ii) due acknowledgment has been made in the text to all other material used

TeamNum
October 15, 2025

Acknowledgments

I would like to thank the following for their efforts with me in my study and for tolerating me during this journey...

Abstract

The abstract of the document is added here...
It is usually written after finishing writing all the other chapters.

Contents

List of Abbreviations

MRSs	Multi-Robot Systems
UAVs	Unmanned Aerial Vehicles
UGVs	Unmanned Ground Vehicles

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Section Name

Some sample text with an Unmanned Aerial Vehicles (UAVs), some citation [**?**, **?**], and some more Unmanned Ground Vehicles (UGVs).

1.2 Another Section

Reference to Section 1.1, and reuse of UGVs nad UAVs with also full use of Multi-Robot Systems (MRSs). Reference to figure 1.1.

Figure 1.1: GUC Logo

Shen *et al.* (2025) [1] introduced **Multi-CAP**, a hierarchical algorithm for multi-robot coverage path planning in unknown environments. The main objective of their approach is to enable multiple robots to cooperatively explore and cover a given area while minimizing redundant paths and maintaining network connectivity among robots.

The **problem is formulated** as an adjacency graph $G = (N, E)$, where N represents the subareas to be covered and E denotes the possible travel paths between them. Each robot starts from a known position and is assigned a sequence of subareas to visit, ensuring that all regions are covered with minimal total travel distance.

The **decision variables** define which subareas are assigned to each robot and in what visiting order. Each route must begin at the robot's starting point and terminate at its final visited subarea, without requiring a return to the start (open route).

The **objective function** aims to minimize the total travel cost of all robots:

$$J(\Pi) = \sum_{m=1}^{M} \sum_{k=0}^{K_m - 1} w_{n_m(k),\, n_m(k+1)} \tag{1.1}$$

where M is the number of robots, K_m is the number of subareas assigned to robot m, and w_{ij} represents the travel cost (distance) between subareas i and j.

The problem is subject to several **constraints**: (1) every subarea must be visited exactly once (coverage constraint), (2) no subarea is assigned to more than one robot (assignment constraint), (3) all robot movements must follow valid edges in E (connectivity constraint), and (4) each robot's path must follow an open-route structure.

Shen *et al.* applied a two-stage heuristic **optimization approach**. In the first stage, a *Nearest-Neighbor heuristic* assigns subareas to robots based on proximity, generating initial feasible routes with minimal overlap. In the second stage, a *2-Opt local search algorithm* refines each route by iteratively adjusting the visiting order to reduce total travel distance.

Finally, each robot performs local coverage within its assigned region using a back and forth sweeping motion or by solving a small local Traveling Salesman Problem (TSP) with the 2-Opt method to ensure complete coverage. This hierarchical framework achieves efficient, connectivity aware exploration while minimizing total path length and coverage time.

In a different study, Mikkelsen *et al.* (2024) [2] presented a novel approach for solving the Multi-Robot Communication-Aware Trajectory Planning (MR-CaTP) problem, which reformulates a general optimization framework by using changes in robot positions as decision variables and introducing linear constraints to ensure communication performance via the Fiedler value (the second smallest eigenvalue of the graph Laplacian) and collision avoidance.

The **problem is formulated** as a receding horizon optimization over K steps for a swarm of N holonomic robots in $\mathbb{R}^n$, where the communication network is modeled as a time-varying undirected graph $G(t) = (\mathcal{V}, E(t))$ with edge weights $w_{ij}(t)$ representing link quality based on inter-robot distances. The goal is to compute feasible and optimal trajectories that maintain network connectivity while fulfilling task-specific objectives, such as inspection or arbitrary motion with communication insurance.

The **decision variables** are the concatenated sequence of position changes (inputs) over K future steps, $\mathbf{U}^k = (\mathbf{u}^k, \mathbf{u}^{k+1}, \dots, \mathbf{u}^{k+K-1}) \in \mathbb{R}^{KnN}$, where each $\mathbf{u}_i^{k+h}$ defines the change in position for robot i at step $k+h$.

The **objective function** minimizes the sum of a generic cost over the K steps:

$$\min_{\mathbf{U}^k} \sum_{h=0}^{K-1} f(\mathbf{u}^{k+h}), \tag{1.2}$$

where $f(\cdot)$ is task-dependent (e.g., quadratic error to points of interest in inspection tasks or deviation from reference inputs in communication insurance).

The problem is subject to several **constraints**: (1) predicted kinematics: $\mathbf{P}^{k+1} = \mathbf{1}_{K\times 1} \otimes \mathbf{p}^k + \mathbf{B}\mathbf{U}^k$, where $\mathbf{P}^{k+1}$ accumulates future positions and $\mathbf{B}$ is a lower triangular accumulation matrix, (2) input bounds: $\|\mathbf{u}_i^{k+h}\|_p \leq \bar{u}$ for all robots i and steps h, ensuring bounded speed, (3) approximated communication performance: $\mathbf{M}^k\mathbf{U}^k \geq (\underline{\lambda_2} - \lambda_2^k)\mathbf{1}_{K\times 1}$, a linear perturbation-based constraint to keep predicted Fiedler values $\hat{\Lambda}_2^{k+1}$ above a lower bound $\underline{\lambda_2} > 0$ for connectivity, and (4) approximated collision avoidance: $\tilde{\mathbf{C}}^k\mathbf{U}^k \leq \tilde{\mathbf{d}}^k$, a linear inequality based on buffered Voronoi partitions from the Delaunay triangulation to prevent inter-robot overlaps.

Mikkelsen *et al.* applied this framework in a **general optimization approximation** using first-order Taylor expansion for Fiedler value prediction and Voronoi-based linearization for tractability, enabling significant computational speedup while remaining close to optimal solutions. In the inspection task application, points of interest are first assigned via minimum perfect matching (solved with the Hungarian algorithm), then trajectories are optimized with a quadratic objective incorporating error to assigned points and Fiedler gradient for relays. In the communication insurance service, the objective minimizes deviation from desired reference inputs, with optional slack variables to soften the Fiedler bound when strict feasibility is challenging. Simulations validate that the Fiedler constraint ensures connectivity at all times, with approximations reducing solve times effectively.

Furthermore, Kashyap *et al.* (2023) [3] introduced a hybrid Lévy Flight–Particle Swarm Optimization (LF-PSO) algorithm for multi-robot exploration in Urban Search and Rescue (USAR) scenarios. Unlike previous approaches that assume known maps, global positioning, or unrestricted communication, the LF-PSO model operates in unknown, GPS-denied, and communication-limited environments. The algorithm combines Lévy Flight for random, long-range exploration with PSO-based inter-robot repulsion to prevent clustering and improve coverage. Simulation results in a 20×20 m area showed that LF-PSO achieved about 76% area coverage, outperforming standalone PSO and LF methods. The study demonstrates a robust, decentralized approach for autonomous robot coordination in real-world search and rescue tasks.

The **problem is formulated** under assumptions that no initial information about the environment (size or obstacle locations) is known, no global positioning or information about the location of other robots is available, no information about the target or victim's position is known, and communication is only possible when other robots are within a specified communication radius d. Initialization involves randomizing the position and velocity (within the constraints) of the robots in the search space and setting the number of iterations where each iteration represents a time step (e.g., robot at time step t+1 has moved when comparing with respect to robot at time step t). To facilitate comprehensive area coverage, each robot's path is represented as a sequence of waypoints in the two-dimensional search space. Waypoints are interconnected to form a path that guides the robot through the search area. The order in which waypoints are connected determines the robot's trajectory. Each waypoint maintains an exploration status to indicate whether it has been visited by the robot or remains unexplored.

The **decision variables** define the best position of waypoints to achieve most efficient exploration. The main decision variables are derived from the hybrid Lévy Flight - Particle Swarm Optimization (LF-PSO) update equations: $x_i(t), y_i(t)$: Position of robot i at time t. $V_{i,j}(t)$: Velocity of robot i in direction j at time t. s_i : Step size in Lévy Flight movement. $L\theta_i$: Movement direction in Lévy Flight. $R\theta_i$: Repulsion direction from nearby robots. P_{lbx_i}, P_{lby_i} : Local best positions (from Lévy Flight exploration), where local means the decision is only based on the Levi Flight Step Distance, not taking any cooperative information from other robots, which affects their motion such as the repulsion mechanism. P_{gbx_i}, P_{gby_i} : Global best positions (from repulsive PSO component).

The **objective function** aims to maximize the total exploration efficiency (fraction of the area explored by all robots):

$$\text{Maximize } f = \frac{A_{\text{covered}}}{A_{\text{total}}} \tag{1.3}$$

where A_{covered} = Total area explored by all robots collectively, and A_{total} = Total area of the environment.

The problem is subject to several **constraints**: (1) Robots must remain within the map boundaries at all times, (2) Each robot's velocity V_i and step size s_i are bounded by maximum limits: $0 < V_i \leq V_{max}, \quad 0 < s_i \leq s_{max}$, (3) Step Size Limit (Based on the Levy Flight algorithm), where the Lévy flight algorithm is a type of random walk used in optimization and search algorithms that combines frequent short-distance steps with occasional long-distance jumps (also randomizes motion direction L_θ from), (4) Communication between robots occurs only when they are within a communication range d, (5) Obstacles are detected only when within a certain proximity based on the robot's front, left, and right sensors, (6) Movement in the presence of obstacles follows the policy: Turn $\frac{\pi}{2}$ Left/Right if Left Sensor FALSE, Front Sensor TRUE, Right Sensor FALSE; Turn $\frac{\pi}{2}$ Left if Left Sensor FALSE, Front Sensor TRUE, Right Sensor TRUE; Turn $\frac{\pi}{2}$ Right if Left Sensor TRUE, Front Sensor TRUE, Right Sensor FALSE; Turn π if Left Sensor TRUE, Front Sensor TRUE, Right Sensor TRUE; Turn $\frac{\pi}{2}$ Right if Left Sensor TRUE, Front Sensor FALSE, Right Sensor FALSE; Turn $\frac{\pi}{2}$ Left if Left Sensor FALSE, Front Sensor FALSE, Right Sensor TRUE, (7) The repulsion distance D_j between robots 1 and 2 is given by $D_j = \frac{1}{d_1} + \frac{1}{d_2}$, (8) Do not pass by an explored way-point. In mathematical formulation, the robot's motion is governed by: $P_{lbx_j} - x_j = s_j \cos(L\theta_j)$, $P_{lby_j} - y_j = s_j \sin(L\theta_j)$, $P_{gbx_j} - x_j = D_j \cos(R\theta_j)$, $P_{gby_j} - y_j = D_j \sin(R\theta_j)$. The modified PSO velocity and position update equations are (this should change when we use other optimization algorithms): $V_{i,j}(t+1) = \omega V_{i,j}(t) + p_\omega \text{rand}(P_{lb_j}(t) - x_{i,j}(t)) + n_\omega \text{rand}(P_{gb_j}(t) - x_{i,j}(t))$, $x_{i,j}(t+1) = x_{i,j}(t) + V_{i,j}(t+1)$, where ω is the inertia weight, p_ω and n_ω are cognitive and social coefficients respectively.

Kashyap *et al.* applied a **hybrid LF-PSO optimization approach** that integrates Lévy Flight for exploration with PSO for repulsion-based coordination, updating robot positions and velocities iteratively in a decentralized manner to maximize coverage while adhering to constraints in unknown environments.

In addition, Som *et al.* (2024) [4] proposed a Particle Swarm Optimization (PSO) based approach for relay-node placement in cluster-based wireless sensor networks (WSNs). The problem was formulated as a multi-objective optimization problem that maximizes connectivity between the placed relay nodes (RNs), maximizes the coverage of sensor nodes (SNs) by the placed relay nodes, and minimizes the number of relay nodes deployed. The main challenge was to strategically position RNs from a set of candidate locations to act as cluster heads. This optimized placement approach balances both cost-effectiveness and performance requirements to enable efficient network backbone construction. This relay node infrastructure deployment exemplifies how *meta-heuristic optimization techniques* can address complex coordination challenges in networked systems, where distributed agents must collectively satisfy multiple objectives while maintaining communication constraints—a fundamental problem that parallels the multi-robot path planning and trajectory coordination strategies.

The **problem is formulated** as an application environment where N sensor nodes $S = \{s_1, s_2, \ldots, s_n\}$ are deployed randomly or manually across a monitored region, and M potential positions $P = \{p_1, p_2, \ldots, p_m\}$ are available for relay node placement. The task is to determine which candidate positions should receive relay nodes to form a cluster-based network.

The **decision variables** include : δ_i indicating whether position p_i is selected for relay node deployment, ω_i indicating whether sensor node s_i is covered by at least one relay node within communication range, ϕ_{ij} indicating connectivity between relay nodes r_j and r_i within communication distance, and α_i indicating whether relay node r_i can directly reach the base station.

There are three **objective functions**. The **first objective function** minimizes

$$F_1 = \frac{1}{M}\sum_{i=1}^{M} x_i \tag{1.4}$$

to reduce the total number of deployed relay nodes and associated costs. The **second objective function** maximizes

$$F_2 = \sum_{\forall s_i \in S} Cov(s_i) \tag{1.5}$$

to ensure comprehensive sensor coverage across the monitored region, where $Cov(s_i)$ represents the set of relay nodes within communication range of sensor node s_i. The **third objective function** maximizes

$$F_3 = \sum_{\forall r_i \in R} Con(r_i) \tag{1.6}$$

to establish robust connectivity among relay nodes forming a reliable backbone toward the base station, where $Con(r_i)$ denotes the set of relay nodes within communication range of r_i and positioned toward the BS. These three objectives are integrated into a composite **fitness function**:

$$\text{Maximize } Fitness = w_1(1 - F_1) + w_2F_2 + w_3F_3 \tag{1.7}$$

where the weights satisfy $\sum w_i = 1$ and $0 \leq w_i \leq 1$ for $i = 1, 2, 3$.

The problem is subject to several **constraints** : (1) coverage constraint ($\omega_n = 1, \forall n, 1 \leq n \leq N$), which ensures that each sensor node has minimum one relay node within its communication range S_{com}, thereby enabling each sensor to be part of a cluster, (2) connectivity constraint ($\sum_{j=1}^{Z}(\phi_{ij} \times \delta_j + \alpha_i) \times \delta_i \geq 1, \forall i, 1 \leq i \leq Z$), which ensures that all deployed relay nodes maintain connectivity with the base station, (3) binary constraint for relay placement ($\delta_i \in \{0, 1\}, \forall i, 1 \leq i \leq M$), which restricts relay node selection to binary values, (4) binary constraints for connectivity and base station reach ($\alpha_i \in \{0, 1\}, \forall i, 1 \leq i \leq Z$ and $\phi_{ij} \in \{0, 1\}, i \neq j, \forall i, j, 1 \leq i \leq Z, 1 \leq j \leq Z$), which ensure that connectivity relationships are strictly defined.

Som *et al.* applied **Particle Swarm Optimization (PSO)** to solve this constrained multi-objective problem, where each particle encodes a candidate relay placement configuration. The PSO algorithm iteratively updates particle velocities and positions guided by personal and global best solutions, refining placements through a balance of exploration and exploitation. Simulation results across two network scenarios demonstrated that PSO selected fewer relay nodes than the Gravitational Search Algorithm while maintaining complete coverage and connectivity, validating its effectiveness for communication-aware relay node deployment.

Finally, Huang *et al.* (2020) [5] addressed a critical gap in multi-robot coverage path planning (MCPP) research by proposing an algorithm specifically designed for outdoor environments with multiple land cover types. This work builds upon and extends existing MCPP methodologies to account for environmental variation, that was overlooked in prior research.

The **problem is formulated** as maximizing the coverage of a rectangular area D representing the search and rescue region, and s robots $\mathcal{R} = \{R_1, R_2, \ldots, R_s\}$ with initial positions $\mathcal{S}_p = \{S_{p_1}, S_{p_2}, \ldots, S_{p_s}\}$. The environment contains multiple land cover types C_v (e.g., farm, road, grass, wood, bare land, Gobi). Each land cover affect the robot performance:

- Visual field $Fd_{C_{v_i}}$: the detection range perpendicular to movement direction
- Moving speed $Sd_{i,r,c}$: the velocity when traversing cell $C_{i,r,c}$

To accommodate these varying capabilities, we decompose the task area using a hierarchical quadtree structure with l layers, where:

$$l = 1 + \log_2\left(\frac{Fd_{\max}}{Fd_{\min}}\right) \tag{1.8}$$

The quadtree construction follows a two-phase process:

Phase 1: Top-down decomposition. Starting from the top layer with cell size $Sc_1 = 2 \times Fd_{\max}$, each subsequent layer i has cells of size $Sc_i = Sc_{i-1}/2$, creating cells $C_{i,r,c}$ at all layers.

Phase 2: Bottom-up integration. To eliminate redundant coverage, cells are classified as either *active* (assigned a land cover type) or *auxiliary* (set to null). Starting from the bottom layer l:

- Each cell $C_{l,r,c}$ is assigned the dominant land cover type within its boundaries
- For layers $i = l-1$ to 1: If four child cells share the same non-null cover type AND their size is smaller than the robot's visual field in that terrain, they are integrated upward by:

$$C_{v_{i,r,c}} = C_{v_{i+1,r*2,c*2}} \text{ (parent inherits type)} \tag{1.9}$$

$$C_{v_{i+1,r*2,c*2}} = C_{v_{i+1,r*2+1,c*2}} = \cdots = \text{null (children deactivated)} \tag{1.10}$$

This ensures all active cells $\mathcal{C}_{\text{active}} = \{C_{i,r,c} : C_{v_{i,r,c}} \neq \text{null}\}$ cover D exactly once, adapting cell sizes to local terrain complexity. To enable efficient task assignment, we establish adjacency relationships using shared neighbor directions $Dn_{i,r,c} \in \{0, 1, \ldots, 15\}$ encoded in binary, where each bit represents a cardinal direction. For cells in different layers, shared neighbors are identified through logical AND operations on direction values.

The **decision variables** include :

- A_j: Area assigned to robot R_j
- P_j: Coverage path for robot R_j

The **objective function** minimizes the maximum task cost across all robots:

$$\min \max_{j \in [1,s]} \{T_j\} \tag{1.11}$$

where the task cost Qa_j for robot R_j is:

$$Qa_j = \sum_{C_{i,r,c} \in A_j} \frac{2Sc_i}{Sd_{i,r,c}} \tag{1.12}$$

The problem is subject to several **constraints**:

$$\bigcup_{j=1}^{s} A_j = D \quad \text{(Complete coverage)} \tag{1.13}$$

$$A_j \cap A_k = \emptyset, \quad \forall j,k \in [1,s], j \neq k \quad \text{(No overlap)} \tag{1.14}$$

$$|T_1| \approx |T_2| \approx \cdots \approx |T_s| \quad \text{(Balanced workload)} \tag{1.15}$$

$$St_j \text{ is connected}, \quad \forall j \in [1,s] \quad \text{(Continuous)} \tag{1.16}$$

$$S_{p_j} \in A_j, \quad \forall j \in [1,s] \quad \text{(Initial position)} \tag{1.17}$$

Chapter 2

Methodology

Problem Formulation

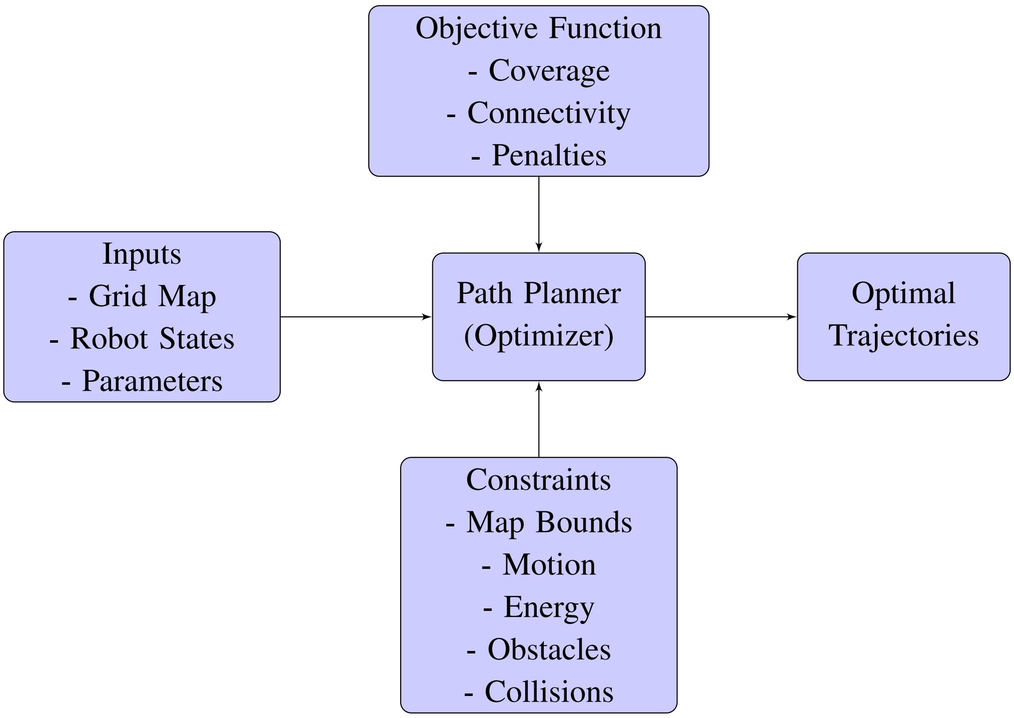

Figure 2.1: High-level overview of the path planning optimization approach.

This chapter presents the mathematical formulation of the multi-robot exploration problem with distance-weighted connectivity constraints and energy limitations. We model the environment as a discrete grid and formulate the problem as a constrained optimization that balances area coverage with maintaining communication links between robots, while accounting for battery constraints and penalizing undesirable behaviors such as network disconnection and obstacle encounters.

Assumptions

The environment is represented as a grid map $M \in \{0, 1, 2, 3\}^{n \times m}$ with unit cells of size 1×1. Each cell $M[x, y]$ takes values: $M[x, y] = 0$ means unexplored, $M[x, y] = 1$ means free space, $M[x, y] = 2$ means obstacle, and $M[x, y] = 3$ marks a cell occupied by a robot.

We assume that all robots are exactly identical in their capabilities, including sensing range, communication radius, motion constraints, and battery capacity. This makes coordination simpler and ensures fairness in task allocation, as no robot has advantages over others.

We assume that obstacles are unknown at the start, which matches realistic exploration scenarios where the environment structure is discovered gradually. As robots explore, newly detected obstacles are shared among all robots through their communication network. Let $O_t \subseteq \{(x, y) \mid 1 \leq x \leq n, 1 \leq y \leq m\}$ denote the set of known obstacle locations at time t, with $O_t \subseteq O_{t+1}$ for all $t \geq 0$. This monotonic growth of obstacle knowledge captures the fact that once an obstacle is discovered, it remains known. When a robot encounters an obstacle during execution, it uses a reactive obstacle avoidance strategy to navigate around the obstruction while trying to maintain its overall trajectory toward the planned goal.

Robots are modeled as point agents that move synchronously at discrete time steps $t \in \{0, 1, 2, \ldots\}$. Motion follows a 4-connected grid topology (up, down, left, right) or allows robots to stay in place. This discretization simplifies planning while maintaining sufficient capability for typical robotic platforms.

Each robot is equipped with a battery that permits movement for a maximum of $\ell \in \mathbb{N}$ cells, where $\ell > 0$ represents the energy budget. Since all robots are identical, they share the same energy capacity. This constraint reflects the practical limitation that robots must complete their exploration mission within their available energy resources.

We assume robots share their positions at each time step through local communication. The desirability of maintaining communication between any pair of robots is modeled by a weight function that decreases with their Euclidean distance, reflecting the fact that closer robots have more reliable communication links.

Map, Robots, and Positions

There are $N \in \mathbb{N}$ robots indexed by $i \in \{1, \ldots, N\}$. The position of robot i at time t is denoted

$$p_{i,t} = (x_{i,t}, y_{i,t}) \in \mathbb{Z}^2,$$

where

$$x_{i,t} \in \{1, \ldots, n\}, \qquad y_{i,t} \in \{1, \ldots, m\}.$$

Initial positions $p_{i,0}$ are given for all $i \in \{1, \ldots, N\}$.

The discrete grid representation allows us to formulate the problem as a mixed-integer program. Each robot's state at any time is fully described by its integer coordinates within the map boundaries.

The 4-connected motion constraint is enforced by the Manhattan distance:

$$|x_{i,t} - x_{i,t-1}| + |y_{i,t} - y_{i,t-1}| \leq 1, \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 1.$$

This constraint ensures that each robot can move at most one grid cell per time step in one of the four cardinal directions, or remain stationary. The Manhattan distance metric naturally captures the 4-connected neighborhood structure and prevents diagonal movements, which simplifies motion planning and collision avoidance.

Energy Consumption Model

To model battery constraints, we define the cumulative distance traveled by robot i up to time t as:

$$D_{i,t} = \sum_{\tau=1}^{t} \left(|x_{i,\tau} - x_{i,\tau-1}| + |y_{i,\tau} - y_{i,\tau-1}|\right), \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 1.$$

Each unit of movement (one grid cell) consumes one unit of battery. The energy constraint for each robot is:

$$D_{i,t} \leq \ell, \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 0,$$

where $\ell \in \mathbb{N}$ is the maximum number of cells each robot can traverse.

This formulation ensures that the total distance traveled by any robot never exceeds its battery capacity. The constraint applies at every time step, meaning that robots must stop moving once they have exhausted their energy budget. Since robots are identical, they all share the same energy limit ℓ. This constraint forces the planner to balance exploration goals with energy availability, potentially requiring some robots to remain stationary or return to already-explored areas to conserve energy for critical coordination tasks. The mission effectively ends when all robots have either exhausted their batteries or completed the exploration task.

Coverage and Distance-Weighted Connectivity

Coverage is a fundamental metric in exploration tasks, measuring the fraction of the environment that has been visited by at least one robot. We define the binary coverage variable

$C_{x,y} \in \{0, 1\}$ for each cell (x, y):

$$C_{x,y} = \begin{cases} 1, & \text{if } \exists\, i \in \{1, \dots, N\},\ \exists\, t \geq 0 \text{ such that } p_{i,t} = (x, y), \\ 0, & \text{otherwise.} \end{cases}$$

The coverage is enforced via indicator constraints:

$$C_{x,y} \ \geq\ \mathbb{I}[p_{i,t} = (x, y)]\,, \quad \forall x \in \{1, \dots, n\},\ \forall y \in \{1, \dots, m\},\ \forall i \in \{1, \dots, N\},\ \forall t \geq 0,$$

where $\mathbb{I}[\cdot]$ is the indicator function that equals 1 when its argument is true and 0 otherwise.

These constraints ensure that once a cell is visited by any robot at any time, its coverage variable is set to 1 and remains 1 thereafter. This formulation naturally handles the cumulative nature of coverage: a cell needs to be visited only once to be considered covered, regardless of whether robots later leave or return to it.

To model communication quality between robots, we first define the Euclidean distance between robots i and j at time t as:

$$d_{ij,t} \ =\ \sqrt{(x_{i,t} - x_{j,t})^2 + (y_{i,t} - y_{j,t})^2} \in \mathbb{R}_{\geq 0}.$$

Building on this, we define the distance-weighted link quality $W_{ij,t} \in [0, 1]$ as:

$$W_{ij,t} \ =\ \begin{cases} \dfrac{1}{1 + d_{ij,t}}, & \text{if } d_{ij,t} \leq R_c, \\ 0, & \text{if } d_{ij,t} > R_c, \end{cases}$$

where $R_c > 0$ is the communication radius threshold.

This formulation captures two important aspects of real-world communication: (1) link quality degrades continuously with distance within communication range, following an inverse relationship, and (2) beyond a critical distance R_c, communication becomes impossible. The weight $W_{ij,t}$ reaches its maximum value of 1 when robots are at the same location ($d_{ij,t} = 0$) and decreases smoothly as distance increases, encouraging the optimizer to keep robots reasonably close while exploring.

Network Connectivity and Disconnection Penalty

Define the edge set at time t as:

$$E_t = \left\{ (i, j) \,\middle|\, i, j \in \{1, \dots, N\},\ i < j,\ d_{ij,t} \leq d_{\text{threshold}} \right\}, \quad \forall t \geq 0,$$

where $d_{\text{threshold}} > 0$ is the connectivity distance threshold.

Let $G_t = (V, E_t)$ be the communication graph at time t, where $V = \{1, \ldots, N\}$ is the set of robot indices. We define $\mathcal{C}_t$ as the set of connected components in G_t. For each robot i, let $C_t(i)$ denote the connected component containing robot i at time t.

To measure network disconnection, we introduce a binary indicator variable:

$$\delta_{i,t} = \begin{cases} 1, & \text{if robot } i \text{ is disconnected from the main network at time } t, \\ 0, & \text{otherwise,} \end{cases}$$

where the "main network" is defined as the largest connected component in G_t (breaking ties arbitrarily). Formally:

$$\delta_{i,t} = \mathbb{I}\left[C_t(i) \neq \arg\max_{C \in \mathcal{C}_t} |C| \right], \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 0.$$

When a robot is disconnected, we measure its distance to the main network. Let $V_{\text{main},t} \subseteq V$ denote the set of robots in the main network at time t. The distance from disconnected robot i to the main network is:

$$d_{i,\text{net},t} = \min_{j \in V_{\text{main},t}} d_{ij,t}, \qquad \forall i \in \{1, \ldots, N\} \text{ with } \delta_{i,t} = 1.$$

Since the mission continues until all robots exhaust their energy or complete exploration, we compute the total disconnection penalty by summing over all robots and all time steps up to the maximum mission duration $T_{\max}$:

$$P_{\text{disconnect}} = \sum_{t=0}^{T_{\max}} \sum_{i=1}^{N} \delta_{i,t} \cdot d_{i,\text{net},t},$$

where $T_{\max} = \max\{\ell, nm\}$ is an upper bound on mission duration (either all energy is depleted or all cells are explored).

This penalty term has two important properties: (1) it is zero when all robots remain connected throughout the mission, and (2) when disconnection occurs, the penalty scales linearly with how far the isolated robot(s) are from the main network. This encourages the planner to either maintain full connectivity or, if temporary disconnection is unavoidable, to minimize both the duration and severity (distance) of the disconnection. Robots that are only slightly beyond communication range incur smaller penalties than those that have wandered far from the team.

Obstacle Encounter Penalty

To discourage robots from planning paths that lead to obstacle encounters, we define an obstacle encounter indicator:

$$\eta_{i,t} = \begin{cases} 1, & \text{if robot } i \text{ encounters an obstacle at time } t, \\ 0, & \text{otherwise.} \end{cases}$$

An obstacle encounter occurs when a robot plans to move to a cell that is later discovered to contain an obstacle. Formally, robot i encounters an obstacle at time t if:

$$p_{i,t} \in O_t \setminus O_{t-1}, \qquad \text{for } t \geq 1.$$

This captures newly discovered obstacles that were unknown during planning. When such an encounter occurs, the robot must reactively navigate around the obstacle, leading to suboptimal paths, wasted energy, and delays.

The total obstacle encounter penalty is:

$$P_{\text{obstacle}} = \sum_{t=1}^{T_{\max}} \sum_{i=1}^{N} \eta_{i,t}.$$

This penalty counts the total number of obstacle encounters across all robots and all time steps. While it is impossible to completely avoid obstacles that are initially unknown, this penalty term encourages the planner to favor paths through regions with lower perceived obstacle density (based on prior exploration) and to coordinate robot movements such that scouts discover obstacles ahead of the main group. The penalty also indirectly encourages more conservative exploration strategies that reduce the likelihood of unexpected collisions.

Objective Function

The optimization problem seeks to maximize a composite objective that balances exploration, connectivity, energy efficiency, and safety:

$$\max\ f = \underbrace{\alpha \sum_{x=1}^{n} \sum_{y=1}^{m} C_{x,y}}_{\text{coverage term}} + \underbrace{\beta \sum_{t=0}^{T_{\max}} \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} W_{ij,t}}_{\text{distance-weighted connectivity term}} - \underbrace{\gamma \cdot P_{\text{disconnect}}}_{\text{disconnection penalty}} - \underbrace{\zeta \cdot P_{\text{obstacle}}}_{\text{obstacle encounter penalty}},$$

where $\alpha, \beta, \gamma, \zeta \in \mathbb{R}_{>0}$ are positive weighting parameters.

The first term rewards exploring new areas by counting the total number of covered cells. The second term encourages robots to maintain proximity throughout the mission by accumulating link quality weights over all time steps and all robot pairs. The third term penalizes network fragmentation, with penalties scaled by the distance of disconnected robots from the main network. The fourth term discourages obstacle encounters, which lead to inefficient reactive behaviors.

The parameters $\alpha, \beta, \gamma, \zeta$ allow us to tune the relative importance of these competing objectives. Larger α prioritizes aggressive exploration potentially at the cost of team fragmentation and safety. Larger β keeps the team cohesive but may slow down exploration. Larger γ enforces stricter connectivity requirements, particularly penalizing scenarios where robots become isolated far from the team. Larger ζ promotes more cautious exploration strategies that reduce collision risk. The choice of these parameters depends on mission requirements: search-and-rescue operations might prioritize coverage (α) and connectivity (β, γ), while mapping missions in hazardous environments might additionally emphasize safety (ζ).

Constraints

The optimization is subject to several constraints that ensure feasible and safe robot trajectories.

1. **Map bounds:**

$$x_{i,t} \in \{1, \ldots, n\}, \qquad y_{i,t} \in \{1, \ldots, m\}, \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 0.$$

   These constraints ensure that all robots remain within the boundaries of the defined map at all times. Violating these bounds would place robots in undefined regions outside the environment.

2. **Motion constraint (4-connectivity):**

$$|x_{i,t} - x_{i,t-1}| + |y_{i,t} - y_{i,t-1}| \leq 1, \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 1.$$

   This constraint enforces the kinematic limits of the robots, ensuring that position changes between consecutive time steps are physically realizable. The Manhattan distance bound of 1 permits movement to any of the four adjacent cells or staying in place, but prohibits diagonal moves and larger jumps.

3. **Energy budget:**

$$D_{i,t} = \sum_{\tau=1}^{t} \left(|x_{i,\tau} - x_{i,\tau-1}| + |y_{i,\tau} - y_{i,\tau-1}|\right) \leq \ell, \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 0.$$

   This constraint ensures that the total distance traveled by each robot at any time does not exceed its battery capacity ℓ. Since all robots are identical, they share the same energy limit. This forces the planner to make strategic decisions about which regions to explore and which robots should undertake longer journeys versus remaining in supporting roles. Once a robot reaches its energy limit, it must remain stationary for the remainder of the mission.

4. **Obstacle avoidance (with online discovery):**

$$p_{i,t} \notin O_t, \qquad \forall i \in \{1, \ldots, N\},\ \forall t \geq 0,$$

where $O_t \subseteq O_{t+1}$ for all $t \geq 0$.

These constraints prevent robots from planning to occupy cells containing known obstacles. Since obstacles are discovered gradually ($O_t \subseteq O_{t+1}$), the constraint set grows over time as exploration proceeds. This formulation naturally handles the online nature of the problem: robots must avoid all obstacles known at time t when planning their position at time t, but cannot anticipate obstacles that will only be discovered later. When a robot encounters a previously unknown obstacle during execution (triggering the obstacle encounter penalty P_{obstacle}), it uses reactive obstacle avoidance to navigate around the obstruction.

5. **Collision avoidance:**

$$p_{i,t} \neq p_{j,t}, \qquad \forall i, j \in \{1, \ldots, N\} \text{ with } i \neq j,\ \forall t \geq 0.$$

This constraint ensures that no two robots occupy the same cell simultaneously, preventing inter-robot collisions. Combined with the motion constraints, this also implicitly prevents certain types of edge collisions where robots would swap positions in a single time step.

6. **Coverage update:**

$$C_{x,y} \ \geq \ \mathbb{I}[p_{i,t} = (x, y)]\,, \quad \forall x \in \{1, \ldots, n\},\ \forall y \in \{1, \ldots, m\},\ \forall i \in \{1, \ldots, N\},\ \forall t \geq 0.$$

These constraints link the coverage variables to the robot positions, ensuring that the coverage term in the objective function accurately reflects the set of visited cells. Since $C_{x,y}$ appears with a positive coefficient in the objective and we are maximizing, the optimizer will set $C_{x,y} = 1$ whenever any robot visits cell (x, y) at any time.

Decision Variables

The optimization determines the complete trajectory of each robot from the initial time until either all energy is exhausted or exploration is complete:

$$\left\{ p_{i,t} = (x_{i,t}, y_{i,t}) \;\middle|\; i \in \{1, \ldots, N\},\ t \in \{0, 1, 2, \ldots, T_{\text{max}}\} \right\},$$

where $T_{\text{max}} = \max\{\ell, nm\}$ is an upper bound on mission duration.

Given the initial positions $\{p_{i,0}\}_{i=1}^{N}$ and the current obstacle knowledge O_0, the optimization must select positions for all robots at all future time steps that satisfy all constraints while maximizing the objective function. This results in a total of $2NT_{\text{max}}$ integer decision variables (two coordinates per robot per time step), plus nm binary coverage variables, plus NT_{max} binary disconnection indicators $\delta_{i,t}$, plus NT_{max} binary obstacle encounter indicators $\eta_{i,t}$, making this a challenging mixed-integer nonlinear optimization problem due to the distance computations in the connectivity and penalty terms.

The planner generates the full path for each robot from start to finish in one optimization solve. This approach differs from receding horizon methods and provides globally optimal paths subject to the known information at planning time. However, since obstacles are discovered online, the actual executed paths may deviate from the plan when reactive obstacle avoidance is triggered. In practice, replanning may be performed periodically as new obstacle information becomes available, but each planning cycle produces a complete path rather than just the next few steps.

Chapter 3

Results

Chapter 4

Conclusion

Chapter 5

Future Work

Appendix

Bibliography

[1] Zongyuan Shen, Burhanuddin Shirose, Prasanna Sriganesh, Bhaskar Vundurthy, Howie Choset, and Matthew Travers. Multi-cap: A multi-robot connectivity-aware hierarchical coverage path planning algorithm for unknown environments, 2025.

[2] Jeppe Heini Mikkelsen, Roberto Galeazzi, and Matteo Fumagalli. Optimal multi-robot communication-aware trajectory planning by constraining the fiedler value, 2024.

[3] Gautam Siddharth Kashyap, Deepkashi Mahajan, Orchid Chetia Phukan, Ankit Kumar, Alexander EI Brownlee, and Jiechao Gao. From simulations to reality: enhancing multi-robot exploration for urban search and rescue. *International Journal of Information Technology*, pages 1–12, 2025.

[4] Suman Som, Marlom Bey, and Pratyay Kuila. Particle swarm optimization for efficient placement of relay nodes in cluster-based wireless sensor networks, 10 2024.

[5] Xiang Huang, Min Sun, Hang Zhou, and Shuai Liu. A multi-robot coverage path planning algorithm for the environment with multiple land cover types. *IEEE Access*, 8:198101–198117, 2020.